

CyberSOC Platform

API Gateway & Authentication Architecture

Version 1.0 | 2026-08-24

INTERNAL - TECHNICAL
SPECIFICATION

Document Purpose: This specification defines the complete API Gateway and Authentication architecture for the CyberSOC AI-Native Security Operations Center Operating System. It covers OAuth 2.0/OIDC implementation, JWT token management, rate limiting strategies, service mesh integration, and security controls required for production deployment.

Target Audience: Platform Architects, Backend Developers, DevOps Engineers, Security Engineers, SRE Team

Go-Live Phase: Phase 1B - Critical Path Item (Authentication & API Infrastructure)

TABLE OF CONTENTS

- 1. Executive Summary.....3
- 2. Architecture Overview.....4
 - 2.1 Design Principles.....4
 - 2.2 Component Architecture.....5
 - 2.3 Technology Stack.....6
- 3. OAuth 2.0 / OIDC Implementation.....7
 - 3.1 Authorization Flows.....7
 - 3.2 Token Management.....9
 - 3.3 Identity Provider Integration.....11
- 4. API Gateway Architecture.....13
 - 4.1 Gateway Configuration.....13
 - 4.2 Request Processing Pipeline.....15
 - 4.3 Plugin System.....17
- 5. Rate Limiting Strategy.....19
 - 5.1 Rate Limiting Algorithms.....19
 - 5.2 Implementation Configuration.....21
 - 5.3 Quota Management.....23
- 6. Service Mesh Integration.....25
 - 6.1 Mesh Topology.....25
 - 6.2 Traffic Management.....26
 - 6.3 Security Policies.....27
- 7. Security Controls.....29
 - 7.1 Threat Mitigation.....29
 - 7.2 CORS & CSP Policies.....31
 - 7.3 WAF Rules.....32
- 8. Monitoring & Observability.....34
 - 8.1 Metrics Collection.....34
 - 8.2 Distributed Tracing.....35
 - 8.3 Alerting Rules.....36
- 9. Implementation Roadmap.....37
- Appendix A: Configuration Reference.....39
- Appendix B: Error Response Schema.....41
- Appendix C: Security Checklist.....42

1. EXECUTIVE SUMMARY

The CyberSOC API Gateway and Authentication Architecture forms the critical security perimeter and traffic management layer for the entire platform. This specification defines how all external requests are authenticated, authorized, rate-limited, and routed to backend microservices, while maintaining the performance, reliability, and security standards required for enterprise-grade security operations. The architecture implements industry-standard protocols including OAuth 2.0, OpenID Connect, and JSON Web Tokens (JWT) to ensure interoperability with existing identity infrastructure while providing the granular access controls necessary for a multi-tenant SOC platform.

1.1 Scope and Objectives

This specification addresses four primary architectural domains essential to platform security and scalability. First, the Identity and Access Management domain establishes secure authentication mechanisms supporting multiple identity providers, multi-factor authentication enforcement, and fine-grained session management across web, mobile, and API client types. Second, the API Gateway domain provides centralized request handling with protocol translation, request/response transformation, caching, and intelligent routing based on tenant context, user permissions, and system load conditions. Third, the Traffic Control domain implements sophisticated rate limiting, quota management, and abuse prevention mechanisms that protect backend services from overload while ensuring fair resource allocation among tenants and user segments. Fourth, the Observability domain delivers comprehensive monitoring, distributed tracing, and security analytics capabilities that enable operational teams to detect anomalies, troubleshoot issues, and demonstrate compliance with regulatory requirements.

1.2 Key Requirements

Requirement Category	Specific Requirement	Priority	Success Metric
Authentication Security	Support OAuth 2.0 + OIDC with multiple IDPs; MFA enforcement; token rotation; session hijacking prevention	Critical	Zero authentication bypass vulnerabilities; MFA effectiveness >99.9%
Authorization Granularity	RBAC with 47+ permissions; tenant isolation; attribute-based access control	High	100% API endpoint coverage; zero cross-tenant data leakage
Performance	P50 latency <20ms gateway overhead; P99 latency <100ms; support 10k RPS per instance	High	99.9% SLO achievement; no service degradation during peak load
Availability	99.95% uptime SLA; graceful degradation; no single point of failure	High	Successful failover testing; 0 data loss during maintenance
Scalability	Horizontal scaling; stateless design; handling 100k+ concurrent users	Medium	Linear scaling validation; capacity planning accuracy >95%
Observability	Complete request tracing; metrics export; security event logging	Medium	MTTR <15min for auth issues; audit completeness >99%
Compliance	SOC 2 Type II aligned; GDPR data protection; FedRAMP Ready	High	Annual audit passage; penetration test findings resolved

1.3 Design Decisions

Several key architectural decisions shape this specification, each representing a deliberate trade-off between competing concerns. The selection of Kong Gateway as the primary API gateway technology prioritizes plugin extensibility, community support, and Kubernetes-native deployment over alternatives

like Ambassador or AWS API Gateway that would introduce vendor lock-in or reduced customization capability. The decision to implement stateless JWT-based authentication with short-lived access tokens (15 minutes) and longer-lived refresh tokens (7 days) balances security (reduced token lifetime limits exposure window) against usability (refresh tokens enable persistent sessions without frequent re-authentication). The choice of Redis as the shared state store for rate limiting counters, token blacklists, and session metadata enables horizontal scaling of gateway instances while maintaining consistent state across the cluster.

2. ARCHITECTURE OVERVIEW

The API Gateway and Authentication system follows a layered architecture pattern, separating concerns across edge infrastructure, gateway processing, service mesh communication, and backend application logic. Each layer has well-defined responsibilities and interfaces, enabling independent scaling, testing, and evolution of components without impacting other layers. This section describes the overall architecture, component relationships, and technology selections that enable the system to meet its functional and non-functional requirements.

2.1 Design Principles

Principle	Description	Application in This Architecture
Defense in Depth	Multiple security layers so that compromise of one layer does not result in system failure	Edge WAF + Gateway Auth + Service Mesh TLS + Application Auth
Zero Trust	Never trust implicitly; verify every request and communication	External API calls authenticated, service-to-service calls use mTLS; no implicit trust
Fail Secure	System defaults to secure state on failure	Auth fails deny/block requests, rate limiter defaults to restrictive mode
Minimal Privilege	Components and users have only minimum required permissions	Scoped JWT claims, RBAC for gateway functions, backend service accounts
Separation of Concerns	Each component handles one aspect of the system	Gateway handles routing/auth/rate limit; Mesh handles identity; app handles business logic
Statelessness	Gateway instances maintain no session state	Stateful stateless with distributed Redis (session DB) instance replacement safe

2.2 Component Architecture

The system comprises eight major component groups arranged in logical layers from the network edge inward. At the edge layer, CDN and DDoS protection services filter malicious traffic before it reaches application infrastructure. The ingress layer manages TLS termination, initial routing, and basic geo-access controls. The gateway layer performs core API management functions including authentication, authorization, rate limiting, and request transformation. The service mesh layer handles secure inter-service communication with mutual TLS, traffic splitting, and retry logic. Finally, the application layer contains the business logic microservices that implement CyberSOC functionality.

Layer	Component	Technology	Responsibility	Scaling Model
Edge	CDN/Static Cache	CloudFlare Enterprise	Static asset delivery; DDoS absorption; Global geographic distribution	Global scale, autoscale
Edge	WAF	CloudFlare WAF + ModSecurity (OpenWAF)	Malicious traffic protection; custom rule engine with CDN patching; request scrubbing	Global scale, autoscale
Ingress	TLS Termination	Kubernetes Ingress (NGINX)	Certificate provisioning (Let's Encrypt); TLS termination; HTTP/2	Cluster scale, autoscale
Ingress	Load Balancer	Cloud LB (GCP/AWS) or MetalLB (on-prem)	Weighted distribution; connection draining; failover	Cluster scale, autoscale
Gateway	API Gateway	Kong Gateway (Enterprise OSS)	Auth validation; rate limiting; request transformation; plugin execution	HAAS (2-50 pods), autoscale
Gateway	Identity Proxy	Keycloak (or commercial IDP)	OAuth/OIDC flows; MFA orchestration; User directory sync (if needed)	HAAS (2-10 pods), autoscale
Mesh	Control Plane	Istiod (Istio)	Service discovery; certificate issuance; Configuration distribution	Single HA instance, autoscale

Layer	Component	Technology	Responsibility	Scaling Model
Mesh	Data Plane	Envoy sidecars (Istio)	Inter-service mTLS; traffic management; circuit breaker; observability	Per pod/sidecar
Application	Backend Services	Go/Python/Node.js microservices	Business logic; data access; domain-specific logic	Independent scaling per service

2.3 Technology Stack Justification

Component	Selected Technology	Alternatives Considered	Rationale for Selection
API Gateway	Kong Gateway (OSS/Enterprise)	Amazon API Gateway, Traefik, AWS API Gateway	Open source; Kubernetes-native; Lua/plugin extensibility
Identity Provider	Keycloak (with HA PostgreSQL)	Azure AD, Okta, Azure AD (as OpenID Connect)	Self-hosted; OAuth2/OAuth2; customizable UI; support for multiple realms
Service Mesh	Istio (with Envoy)	Linkerd, Consul Connect, Emissary	Feature-complete; Envoy data plane maturity; K8s integration
State Store	Redis Cluster (mode: cluster)	Memcached, Hazelcast, etcd	Rich data structures for rate limiters/token blacklist; pub/sub
Secrets Management	HashiCorp Vault (HA mode)	AWS Secrets Manager, Kubernetes Secrets	Dynamic Secrets; Encryption at rest; audit logging; K8s integration
Certificate Management	cert-manager (K8s native)	Vault PKI, Let's Encrypt Cloud	Automated provisioning/renewal; multiple issuer support

3. OAUTH 2.0 / OIDC IMPLEMENTATION

The CyberSOC Platform implements OAuth 2.0 and OpenID Connect (OIDC) as its primary authentication and authorization framework, enabling secure delegated access for web applications, mobile clients, API consumers, and service accounts. The implementation supports multiple grant types optimized for different client contexts, enforces strict security parameters exceeding RFC recommendations, and integrates with enterprise identity providers through standard federation protocols. This section details the supported authorization flows, token management policies, and identity provider integration patterns.

3.1 Supported Authorization Grant Types

The platform supports four OAuth 2.0 grant types, each designed for specific client types and security contexts. Grant type selection is enforced based on client registration properties, preventing insecure configurations such as using implicit flow for confidential clients or client credentials in browser-based applications. Each flow includes additional security measures beyond the base OAuth specification, including Proof Key for Code Exchange (PKCE) requirement for public clients, signed refresh token rotation, and device binding verification.

Grant Type	Use Case	Security Enhancements	Token Lifetime	Refresh Support
Authorization Code +PKCE	Web Applications (server-side), Mobile Apps, Microservices	- PKCE (S256) mandatory for public clients - Code challenge prevents auth code interception - State parameter validated - Nonce for replay protection	Access: 15 min ID Token: 15 min Code: 60 sec	Yes - Refresh: 7 days Rotation: mandatory Re-use detection: immediate revocation
Implicit (Deprecated)	Legacy browser apps only, no migration path for existing clients	- No migration path for existing clients - Short token lifetime enforced - Strict referrer validation - Planned removal Q4 2026	Access: 5 min ID Token: 5 min	No - Must re-authenticate; Migration to code+PKCE required
Resource Owner Password Grant	First-party trusted clients, legacy sys admin tools	- Rate limited (5/min/IP) - Requires trusted client flag - MFA prompt on new device - Deprecation warning in response	Access: 15 min Refresh: 7 days	New with sensitive context flow; Trusted client only
Client Credentials	Service-to-service; batch jobs; API integrations	- Confidential client only - Scoped to client's permissions - No user context - Audited usage logging	Access: 30 mins (only ID token)	No - Re-authenticate; Shorter TTL for high-privilege scopes
Device Authorization	TV/apps limited input; IoT devices, headless systems	- User codes, refreshless - Polling interval: 5 sec (max) - Expiration: 10 minutes - Rate limited polling	Access: 15 min (user-dependent)	Yes - Standard refresh; Device binding optional

3.2 JWT Token Structure and Management

JSON Web Tokens (JWT) serve as the primary credential format for authenticated sessions within the CyberSOC ecosystem. Both access tokens and ID tokens follow the JWT structure with carefully scoped claims that balance information richness against token size and privacy considerations. Refresh tokens are

opaque, server-side-only references stored securely in the identity provider's database, never exposed to client applications in structured form. All tokens incorporate cryptographic signatures using RS256 (RSA Signature with SHA-256) algorithm with key rotation according to NIST SP 800-57 recommendations.

3.2.1 Access Token Claims

Claim	Type	Required	Description	Example Value
iss	string	Yes	Issuer identifier (IdP URL)	https://auth.cybersoc.platform
sub	UUID	Yes	Subject (user ID) - stable identifier	a1b2c3d4-e5f6-7890-abcd-ef1234567890
aud	string[]	Yes	Audience(s) - intended recipients	["cybersoc-api", "cybersoc-web"]
exp	numeric	Yes	Expiration time (Unix timestamp)	1690000000
iat	numeric	Yes	Issued-at time (Unix timestamp)	1689999100
jti	UUID	Yes	Unique token ID for revocation tracking	1a2b3c4d-5e6f-7890-1234-567890abcdef
scope	string	Yes	Space-delimited permission scopes	read:users write:incidents admin:config
tenant_id	UUID	Conditional	Organization context (multi-tenant)	org-12345-abcde
role	string	Conditional	Primary role assignment	security_analyst
amr	string[]	Recommended	Authentication Methods References	["pwd", "mfa"]
auth_time	numeric	Recommended	Time of user authentication	1689999000

3.2.2 Token Lifecycle Management

Token lifecycle management encompasses generation, validation, renewal, and revocation processes that ensure credentials remain secure throughout their useful life. The system implements proactive token rotation for refresh tokens, meaning each refresh operation issues a new refresh token while immediately invalidating the previous one. This detects token theft because legitimate clients will present the newest token while attackers holding stolen older tokens will trigger a reuse detection alert and session termination. Access tokens cannot be revoked individually (stateless nature); instead, sensitive operations require recent authentication (step-up auth) and the system maintains a token blacklist in Redis for emergency revocation with maximum 60-second propagation delay.

Operation	Trigger	Process	Side Effects	Propagation Delay
Token Generation	Successful authentication	IdP creates signed JWT; JTI registered	Login Redis (JTI, token life, scope) updated	Immediate
Token Validation	Each API request to gateway	Signature verified (JWKS cached); Claims validated	Expiration, scope, audience checked	Immediate
Token Refresh	Access token expired; Refresh token valid	New access token issued; Old invalidated	Session extended; Old token blacklisted	Immediate
Token Revocation	User logout; Admin action	JTI added to Redis blacklist	Admin signal to IdP; Session ended	Immediate
Key Rotation	Scheduled (90 days) or Emergency	New RSA key pair generated; JWKS updated	IdP key rotation; Cache purge	60 seconds

3.3 Identity Provider Federation

The CyberSOC Platform integrates with enterprise identity providers through standard federation protocols, enabling organizations to leverage existing identity investments including corporate directories, MFA solutions, and governance tooling. The primary integration mechanism is SAML 2.0 for enterprise SSO scenarios, with OIDC federation for cloud-first identity providers. The platform's bundled Keycloak instance can operate in standalone mode (managing its own user store) or proxy mode (federating to upstream IdPs), with automatic user provisioning and attribute mapping.

Protocol	Use Case	Supported Providers	Configuration Complexity	Features Supported
SAML 2.0	Enterprise SSO; Government/Defense; Healthcare	Azure AD, Okta, Ping Identity, OneLogin, Shibboleth, etc.	Medium to High	Single Sign-On, Attribute Mapping, MFA, Session Persistence
OIDC (Federation)	Cloud-native orgs; DevOps; SaaS Integrations	Google, Microsoft, GitHub, Auth0, Okta, etc.	Low to Medium	OpenID Connect, Social Login, Refresh Tokens
LDAP/AD	On-premise directory sync; Legacy systems	Microsoft Active Directory, OpenLDAP, etc.	Medium	Directory Synchronization, Password Sync
Social Login	Consumer-facing portals; Mobile apps	Google, Facebook, Microsoft, Apple, etc.	Low	OAuth 2.0, OpenID Connect, Social Identity

4. API GATEWAY ARCHITECTURE

The API Gateway serves as the unified entry point for all programmatic access to the CyberSOC Platform, implementing the facade pattern that shields backend services from direct external exposure. Built on Kong Gateway, the gateway provides protocol translation, authentication enforcement, rate limiting, request transformation, caching, and extensive observability hooks. This section details the gateway configuration schema, request processing pipeline, and extensible plugin architecture that enables customization for tenant-specific requirements without modifying core gateway code.

4.1 Gateway Deployment Configuration

The gateway operates in Kubernetes as a Deployment with Horizontal Pod Autoscaler (HPA) responding to CPU utilization (target 70%) and custom metrics (request latency P99, queue depth). A minimum of 2 replicas ensures availability during rolling updates, with maximum replicas configured based on expected peak load plus 30% headroom. Gateway pods connect to a Redis cluster for distributed state (rate limit counters, cache entries, feature flags) and to the PostgreSQL database for configuration persistence and analytics data.

Parameter	Value	Rationale	Tuning Guidance
Worker Processes	auto (matches CPU cores)	Maximize throughput; each worker handles a subset of requests	Monitor CPU utilization; increase with high connection counts
Worker Connections	10000 per worker	Support high concurrency; buffer for burst traffic	Monitor connection counts; reduce if OOM occurs
Upstream Keepalive Pools	64 connections per upstream	Connection reuse reduces TLS handshake overhead	Decrease if backend services are slow to accept connections
Client Body Size	50 MB (configurable per route)	Allow file uploads (PCAP, evidence); limit for PUT/PATCH	Lower for PUT/PATCH; higher for upload endpoints
Client Read Timeout	60 seconds	Long-running queries (analytics, reports) or large responses	Reduce for latency-sensitive routes; increase for analytics
Upstream Connect Timeout	10 seconds	Allow DNS resolution + TCP + TLS handshake	Increase if backends in different region/AZ; decrease if slow
Upstream Send/Read Timeout	16 seconds each	Match client timeout; backend should respond quickly	Align with backend SLAs; shorter timeouts error faster
SSL Protocols	TLS 1.2, TLS 1.3 only	Disable deprecated TLS 1.0/1.1; enforce modern encryption	Remove TLS 1.1 when all clients support 1.3 (recommended)
SSL Ciphers	ECDHE+AESGCM:ECDHE+ChaCha20	Forward secrecy required; prefer AEAD ciphers	Avoid weak ciphers if analytics or security is critical
Access Log Format	JSON (structured)	Machine-parseable for log aggregation and search	Add pipeline fields for correlation; consider compacting
Error Log Level	warn (production)	Balance visibility vs noise; info for debugging, warn for errors	Reduce logging; promote alerts for critical errors
Plugin Bundling	Enabled (enterprise)	Faster startup with pre-bundled plugins	Not applicable to OSS edition; consider for performance

4.2 Request Processing Pipeline

Every incoming request traverses a deterministic sequence of processing phases, each executed by specific plugins configured globally or per-route. The pipeline implements the chain-of-responsibility pattern where each phase can approve the request (pass to next phase), reject it (return error response immediately), or transform it (modify headers/body before passing along). Understanding this pipeline is

essential for debugging issues, adding custom behavior, and optimizing performance by eliminating unnecessary processing for simple requests.

Phase	Position	Plugins Executed	Function	Failure Action
1. SSL Termination	Edge (before Kong)	Nginx SSL module, cert-manager	Decrypt TLS; validate certificate	Terminate with TLS error; handshake failure
2. IP Allowlist/Denylist	Global (pre-auth)	ip-restriction (Kong plugin)	Check source IP against allowlist	Return 403 Forbidden with reason
3. Rate Limit (Global)	Global (pre-auth)	rate-limiting (advanced)	Apply global rate limits (IP-based)	Return 429 Too Many Requests with limit
4. CORS Handling	Global (pre-auth)	cors (Kong plugin)	Validate Origin header; set appropriate CORS headers	Apply CORS headers; handle preflight
5. Authentication	Route-level	jwt (Kong plugin), oauth2, keycloak	Validate credentials; extract identity	Return 401 Unauthorized with WWW-Authenticate
6. Authorization (RBAC)	Route-level	acl (custom plugin)	Check JWT scopes/claims against policy	Return 403 Forbidden; permission denied
7. Rate Limit (Scope)	Route/Consumer	rate-limiting (advanced)	Apply consumer-specific or tenant-specific limits	Return 429 with scope-specific Reason
8. Request Transformation	Route-level	request-transformer, body-parser	Modify headers/query/body parameters	Return 400 Bad Request if transformation fails
9. Upstream Routing	Core Kong	Balancer (round-robin/least-connections)	Select (weight) backend instance; forward request	Apply health check; filter unavailable if no backends
10. Proxy Forwarding	Core Kong	proxy (Nginx)	Forward request to selected upstream	Return 502/504 if upstream fails/timed out
11. Response Transformation	Route-level	response-transformer	Modify response headers/body; add security headers	Return 400 Bad Request if transformation fails
12. Logging & Metrics	Global (post-request)	file-log, http-log, statsd, prometheus	Write access log; emit timing metrics	Log; write failure to error logs if critical

4.3 Plugin Architecture and Custom Plugins

Kong's plugin system provides the primary extension point for gateway behavior, enabling customization without modifying core code. Plugins execute at defined phases (access, header_filter, body_filter, log) and can modify requests, responses, or perform asynchronous actions like calling external services. The CyberSOC deployment includes a mix of community plugins (for standard functionality) and custom plugins (for domain-specific requirements like tenant-aware rate limiting, step-up authentication prompts, and audit logging).

Plugin Name	Type	Scope	Phase	Purpose	Configuration Highlights
jwt	Community	Global/Route	access	Validate JWT signatures; extract claims	secret: /etc/kong/secrets/jwt; claims_to_verify: [exp, nbf]
acl	Custom	Route	access	Enforce RBAC permissions	allow_jwt_scopes: [read:logs, write:logs]; require_role: admin
rate-limiting-advanced	Community	Global/Route/Consumer	access	Multi-dimension rate limiting (IP, user, tenant)	policy: {ttl: 60, window_size: 3600, max_req: 100}; key: ip
correlation-id	Community	Global	access	Generate/propagate X-Request-ID	request_id_generator: uuidv4; header_name: X-Request-ID
request-transformer	Community	Route	access	Add/remove/modify request headers/body	add_headers: {X-Tenant-ID: \$consumer_tenant_id}
response-transformer	Community	Route	header_filter	Modify response headers and body	add_header: X-Content-Type-Options: nosniff; Server: DaVinci
audit-log	Custom	Global	log	Write structured audit events to Elasticsearch	es_host: \$ELASTICSEARCH_HOST; es_port: 9200; index_prefix: audit
tenant-router	Custom	Global	access	Extract tenant from JWT; route to tenant-specific service	tenant_claim: tenant_id; default_tenant: default

Plugin Name	Type	Scope	Phase	Purpose	Configuration Highlights
step-up-auth	Custom	Route	access	Require re-authentication for sensitive operations, such as changing last password	max_age=3600, challenge=1, step_up_auth=1
cache	Community	Route	access/response	Cache GET responses per cache key strategy, respect application	cache_key_strategy=resp_type,application

5. RATE LIMITING STRATEGY

Rate limiting protects the CyberSOC Platform from abuse, ensures fair resource allocation among tenants and users, and prevents cascading failures during traffic spikes. The implementation uses a multi-dimensional approach that applies limits at different granularities (global, per-tenant, per-user, per-API-key) using sliding window algorithms stored in Redis for consistency across gateway instances. This section details the rate limiting algorithms, configuration parameters, and quota management integration with the licensing system.

5.1 Rate Limiting Algorithms

The platform employs three complementary rate limiting algorithms, each suited to different protection objectives. Sliding window logarithmic (SLIW) provides smooth rate limiting without the sharp boundaries of fixed windows, making it ideal for user-facing APIs where occasional bursts are acceptable. Token bucket allows controlled bursting with sustained rate limits, suitable for streaming and real-time data endpoints. Fixed window counter offers simplicity and predictability for billing-aligned quotas where limits reset on calendar boundaries.

Algorithm	Mechanism	Best For	Pros	Cons	Use Case in CyberSOC
Sliding Window Logarithmic (SLIW)	Maintains a sliding log of requests with logarithmic decay	General API endpoints, smooth rate limiting	Handles traffic spikes gracefully, no abrupt resets	Higher complexity, requires Redis for consistency	APIs for threat intelligence feeds, log ingestion
Token Bucket	Bucket holds tokens (refill rate = limit); requests consume tokens	Streaming endpoints, real-time data feeds	Allows controlled bursts, maintains average rate	Requires stateful storage (Redis), more complex	Security event streams, SIEM data ingestion
Fixed Window Counter	Simple counter per window (e.g., hourly/daily)	Billing-aligned quotas, predictable resets	Simple to implement, easy to audit	Can cause abrupt resets, less smooth	Admin console access, report generation

5.2 Rate Limit Configuration by Tier

Rate limits are calibrated per licensing tier to align pricing with resource consumption while preventing any single tenant or user from monopolizing shared infrastructure. Limits are defined for multiple dimensions including requests-per-minute (throughput), concurrent connections (capacity), data volume (bandwidth), and specific high-cost operations (AI analysis, report generation). Administrators can adjust limits within bounds defined by the licensing agreement, with changes taking effect within 60 seconds across all gateway instances.

Dimension	Free Tier	Professional (\$149/mo)	Business (\$299/mo)	Enterprise (\$50K+/yr)	Enforcement Point
API Requests (per minute)	1,000	10,000	50,000	Custom (SLA-negotiated)	Gateway (pre-auth + post-auth)
API Requests (per day)	50,000	500,000	5,000,000	Unlimited	Gateway (daily counter reset)
Concurrent Connections	5	25	100	500	Gateway (connection tracking)
WebSocket Connections	2	10	50	200	Gateway (upgrade tracking)
Search Queries (per minute)	10	100	500	2,000	Gateway (route-specific)
Data Export (per hour)	1 (max 1K rows)	5 (max 10K rows)	25 (max 100K rows)	Unlimited	Gateway + backend validation

Dimension	Free Tier	Professional (\$149/mo)	Business (\$299/mo)	Enterprise (\$50K+/yr)	Enforcement Point
Report Generation (per day)	10	25	100	Unlimited	Backend (job queue throttle)
AI Analysis Calls (per day)	10	500	5,000	Custom	Backend (API key gating)
File Upload (MB per hour)	100	1,000	10,000	100,000	Gateway (body size + backend)
SSO/SAML Logins (per minute)	N/A	20	100	500	IdP (authentication rate limit)
Password Attempts (per 15 min)	5 (lockout)	5 (lockout)	5 (lockout)	Configurable	IdP (brute-force protection)

5.3 Quota Management and Overage Handling

When a tenant approaches or exceeds their allocated quota, the system follows a graduated response sequence designed to provide fair warning before enforcing hard blocks. At 80% utilization, response headers begin including quota usage information (X-Quota-Used, X-Quota-Limit, X-Quota-Reset). At 95%, the system emits administrative alerts and may introduce artificial latency (exponential backoff starting at 50ms) to encourage throttling without hard-blocking legitimate usage. Upon reaching 100%, subsequent requests receive HTTP 429 (Too Many Requests) with Retry-After header indicating when the quota resets. Enterprise customers with negotiated overage terms may exceed limits with per-unit overage charges applied.

Threshold	Response Behavior	Headers Added	Notifications	Admin Actions Available
< 80%	Normal processing; no throttling	None (unless requested via header)	None	View usage dashboard; adjust limits
80% - 94%	Normal processing; informational headers	X-Quota-Used, X-Quota-Limit, X-Quota-Reset	Info X-Quota-Reset, X-Quota-Used (daily digest)	Upgrade discussion
95% - 99%	Normal processing + increased latency (50-200ms)	X-Quota-Used, X-Quota-Limit, X-Quota-Reset	Warning X-Quota-Reset, X-Quota-Used (immediate)	Contact platform for immediate action
100% (Hard Limit)	HTTP 429 Too Many Requests; Retry-After (seconds to reset)	X-Quota-Used, X-Quota-Limit, X-Quota-Reset	Critical X-Quota-Exceeded	Immediate investigation; hotfix deployment
100% (Overage)	HTTP 429 converted to 200 with X-Overage-Overage: Limit Exceeded	X-Overage-Overage: Limit Exceeded	Info X-Overage-Overage: Limit Exceeded	Enterprise billing team; contract amendment

6. SERVICE MESH INTEGRATION

The Istio service mesh provides secure, observable, and resilient inter-service communication within the CyberSOC Platform's Kubernetes deployment. By deploying Envoy proxy sidecars alongside each application pod, the mesh handles transport security (mutual TLS), traffic management (routing, splitting, mirroring), resilience (retries, circuit breaking, timeouts), and observability (distributed tracing, metrics) transparently to application code. This section defines the mesh topology, traffic management policies, and security configuration required for production operation.

6.1 Mesh Topology and Namespace Strategy

The CyberSOC deployment uses a multi-namespace topology with selective mesh injection. Core platform services (API gateway, identity provider, backend microservices) reside in mesh-enabled namespaces with automatic sidecar injection. Data-intensive workloads (Elasticsearch, Kafka, object storage gateways) run in mesh-disabled namespaces to avoid the overhead of double-proxying large data transfers, communicating with mesh services via explicit gateway endpoints. This hybrid approach optimizes performance for bulk data paths while maintaining mesh benefits for API traffic.

Namespace	Mesh Enabled	Contains	mTLS Mode	Rationale
istio-system	No (control plane)	Istiod, Kiali, Prometheus adapter	N/A (control plane)	Infrastructure; no sidecar needed
cybersoc-gateway	Yes (mandatory)	Kong Gateway pods	STRICT	Entry point; must be in mesh for policy enforcement
cybersoc-auth	Yes (mandatory)	Keycloak, LDAP connector	STRICT	Identity services; high security; mTLS required
cybersoc-api	Yes (default)	All backend microservices (SIEM, EDR, SOAR, etc.)	STRICT	Business logic; primary beneficiaries of mesh
cybersoc-data	No (disabled)	PostgreSQL, Elasticsearch, Redis, Kafka, MinIO	DISABLED	Data stores; high throughput; sidecar would be overhead
cybersoc-monitoring	Optional	Grafana, AlertManager, custom dashboards	DISABLED	Observability stack; outbound only; simplicity
cybersoc-batch	No (disabled)	Report generators, ML pipelines	DISABLED	Batch workloads; long-running; initiate via gateway

6.2 Traffic Management Policies

Istio VirtualServices and DestinationRules define how traffic flows between services, enabling canary deployments, blue-green releases, fault injection testing, and intelligent routing based on headers (including tenant context from the gateway). Traffic policies are version-controlled alongside application code and applied via GitOps workflows to ensure reproducibility and auditability.

Policy Type	Applied To	Configuration	Purpose	Rollback Procedure
Canary Release	Backend services (api-engine, analytics)	Weights-based: 90% v1, 10% v2; Safe rollout of new versions	Gradual traffic shift to new version	Shift traffic back to 100% v1; quick rollback
Circuit Breaker	All upstream dependencies (databases, external APIs)	Open on 5 consecutive 5xx errors; Timeout: 30s	Prevent cascading failures	Adjust thresholds; restart service if needed
Timeout	All inter-service calls	Default: 10s; Database: 30s; External: 60s	Prevent hanging requests	Adjust per-service; monitor latency

Policy Type	Applied To	Configuration	Purpose	Rollback Procedure
Retry	Idempotent operations (GET, PUT, PATCH)	Retries: 5; Backoff: exponential; Max duration: 30s	Handle transient failures	Disable retries (set retries: 0)
Tenant-Aware Routing	Multi-tenant services (e.g., /api/*)	Header: X-Tenant-ID -> Subset (tenant-specific)	Isolate tenants	Revert to default routing
Fault Injection	Testing namespace only	Abort: 5% HTTP 500; Delay: 100ms	Simulate failures	Disable fault injection

6.3 Mesh Security Configuration

The service mesh enforces a zero-trust security model where all inter-service communication requires mutual TLS authentication, regardless of network location. Istio's PeerAuthentication and Authorization policies define which identities can communicate, using SPIFFE IDs (SPIFFE Verifiable Identity Documents) derived from X.509 certificates issued by the mesh CA. This approach ensures that even if an attacker gains network access (e.g., container escape, compromised pod), they cannot impersonate valid services or intercept traffic.

Policy	Scope	Configuration	Enforcement	Exceptions
PeerAuthentication (mtls)	Namespace: cybersoc-app	mode: STRICT; gateway: disabled	Enforced	None
PeerAuthentication (cert)	Cluster-wide (all namespaces)	certificates: mesh CA; ttl: 24h	Enforced	None
Authorization (allow-by-default)	Namespace: cybersoc-app	Default ALLOW; Deny rules for sensitive paths	Enforced	None
Authorization (deny-by-default)	Namespace: cybersoc-data	Default DENY; Allow rules for database access	Enforced	None
Authorization (tenant isolation)	Namespaces: cybersoc-app, cybersoc-data	Deny cross-tenant access	Enforced	None

7. SECURITY CONTROLS

Security controls at the API gateway layer represent the first line of defense against web application attacks, API abuse, and data exfiltration. This section details the specific controls implemented for threat mitigation, cross-origin resource sharing policies, content security provisions, and web application firewall rule sets. These controls complement (but do not replace) security measures implemented within backend services and the service mesh.

7.1 Threat Mitigation Controls

The gateway implements targeted mitigations for common attack vectors identified in the OWASP API Security Top 10 and OWASP Web Application Security Top 10. Each control is configured to block or log suspicious activity while minimizing false positives that could impact legitimate users. Controls operate in blocking mode by default for high-confidence detections and logging mode for heuristic checks that require tuning based on observed traffic patterns.

VASP Category	Attack Vector	Mitigation Control	Configuration	Action on Detect
API1: BOLA	Broken Object-Level Authorization (accessing other user's security data)	Tokenization (aggregating + removing security from JWT)	Block (413) B Security	Security audit
API2: Broken Auth	Compromised authentication (JWT validation)	MFA enforcement (signature verification)	Block (401) weeks; MFA	At least one factor
API3: Excess Data Exposure	More data than needed	Response truncation	Field-level filtering per route	TRANSPIDRM (strip fields) unless explicitly
API4: Lack of Rate Limiting	Denial of service (DDoS)	Dimensional rate limiting (e.g. 5 requests per user per minute)	Block (429) rate	Redistributed load
API5: Broken Access Control	Unauthorized API endpoint access (e.g. /admin)	Role-based access control (RBAC)	Specific JWT scopes	Block (403) scope
API6: Mass Assignment	Raw JSON/PATCH request	Schema validation (JSON Schema)	Block (400) Invalid request	Reject
A03: InjectionSQL/NoSQL/LDAP/Command	Input injection	Parameterized queries (escaping)	Block (400) Invalid request	Block
A05: Misconfigured Admin Panels	Exposed admin panels	Rate limiting, IP whitelisting, verbose error sanitization	Block (404/403)	Log endpoints
A07: XSS	Cross-site scripting via reflected output	Content Security Policy (CSP) headers	Block (sanitize)	Security Policy
A09: SSRF	Server-side request forgery	URL validation, allowlist	Block (400) Invalid request	Block

7.2 Cross-Origin Resource Sharing (CORS) Policy

CORS policies control which external origins can make cross-origin requests to the CyberSOC API, protecting against unauthorized embedding and data extraction by malicious websites. The policy distinguishes between browser-based API calls (which enforce CORS preflight) and server-to-server calls (which do not). For embedded dashboard scenarios (iframe integration into customer portals), specific origins are whitelisted with restricted allowed methods and headers.

Scenario	Allowed Origins	Allowed Methods	Allowed Headers	Credentials	Max Age	Notes
Production API (direct)	For specific frontend	GET, POST, PUT, PATCH, DELETE	Content-Type, Authorization	True (for authenticated requests)	86400 (24h)	Standard API access; Credentials required

Scenario	Allowed Origins	Allowed Methods	Allowed Headers	Credentials	Max Age	Notes
Embedded Dashboard (Internal customer portal)	Internal IP ranges only (6000)	GET, POST (JSON only)	Content-Type, Authorization	false (no cookies)	3600 (1h)	Read-only embed; No auth
Webhook Callbacks	N/A (server-to-server, no browser)	POST only	X-Webhook-Signature, X-Webhook-Timestamp	true	Signature-Type	Secure auth; IP
Developer Portal / Docs	docs.cybersoc.platform.io	GET, POST (JSON dev)	Accept, Accept-Language, Content-Type	false	200 (2h)	Public documentation; Limited
Public SDK Examples	codepen.io, jsfiddle.net, StackBlitz.com	GET, POST (JSON)	Content-Type, Authorization	false (user-provided)	3600	Interactive demos; Limited

7.3 Web Application Firewall (WAF) Rule Set

The WAF rule set provides an additional layer of application-layer attack prevention, operating at the edge (CloudFlare) and optionally at the gateway (ModSecurity core rule set). Rules are organized into tiers by severity and confidence, with high-confidence attacks blocked immediately and lower-confidence detections logged for analysis and potential rule tuning. The rule set is managed as code (YAML definitions) and undergoes regular review to address emerging threats and reduce false positive rates based on production traffic analysis.

Rule ID	Category	Pattern Detected	Severity	Action	Paranoia Level	False Positive Risk
SQL Injection (union-based)	Injection	UNION SELECT, UNION ALL SELECT	High	Block (403) + CAPTCHA challenge	PL1 (always on)	Low (clear attack pattern)
SQL Injection (blind/injection)	Injection	SLEEP(), BENCHMARK(), WAITFOR DELAY()	Medium	Block (403) + IP reputation check	PL1 (always on)	Very Low (these functions are not in all DBs)
Remote Code Execution	Injection	system(), exec(), eval(), passthru()	Critical	Block (403) + Immediate Security alert	PL2 (always on)	Low (no legitimate use)
Path Traversal	Injection	../, ..\, %2e%2e/, encoded variants	High	Block (403) + Sanitize path	PL1 (always on)	Medium (some file names)
XSS (Reflected)	XSS	<script>, javascript:, onerror=, onload=	High	Sanitize (escape output)	PL1 (always on)	Medium (rich text editors)
Scanner/Bot Detection	Automated Attacks	User-Agent: sqlmap, nikto, nmap, masscan, etc.	Medium	Challenge (CAPTCHA) then block if failed	PL1 (always on)	Low (legitimate scanners)
Session Fixation	Broken Auth	Session ID in URL parameter; Cookie with path=/	High	Force Secure/HTTPS only	PL1 (always on)	Low (no legit reuse)
Sensitive File Access	Misconfiguration	Requests for .env, .git, .htaccess, etc.	Medium	Block (403) + Log with alert	PL2 (priority)	Very Low (never legit)
Command Injection	Injection	;&\$() ` in parameters with curl, bin, etc.	Critical	Block (403) + External Security incident	PL2 (moderate)	Low (shell metacharacters)
XXE (XML External Entity)	Injection	!ENTITY, !DOCTYPE, SYSTEM, PUBLIC	High	Block (403) + Disable XML parsing for endpoints	PL2 (moderate)	Low (XML rarely used)

8. MONITORING AND OBSERVABILITY

Comprehensive observability is essential for operating the API gateway and authentication system at scale, enabling rapid detection of anomalies, efficient troubleshooting of issues, and demonstration of compliance through audit-quality logs. The monitoring stack collects three signal types: metrics (numerical measurements over time), traces (causal chains of requests across services), and logs (discrete events with context). This section defines what is collected, how it is stored, and how alerts are configured to notify operators of conditions requiring attention.

8.1 Metrics Collection and Dashboards

Metrics are emitted in Prometheus exposition format from Kong plugins, Istio sidecars, and application code, scraped by Prometheus every 15 seconds, and stored with 15-day retention for detailed analysis (aggregated data retained longer). Key metric categories include request volume (counts, rates), latency distributions (histograms), error rates (by type), resource utilization (CPU, memory, connections), and business metrics (active users, license consumption). Grafana dashboards present these metrics with drill-down capability from summary views to individual instance details.

Metric Name	Type	Source	Labels	Dashboard Use	Alert Threshold
kong_http_requests_total	Counter	Kong (statsd exporter)	method, route, service, status_code, upstream	Request volume trends, Error rate % (warning)	Route 4% (warning)
kong_request_latency_seconds	Histogram	Kong (statsd exporter)	route, service (le: 0.005, latency: 0.25, 0.05, 0.01)	Latency P95/P99, Slow requests	P99 > 500ms (info), > 1000ms (warn)
kong_upstream_health	Gauge	Kong (health check plugin)	upstream, target, zone	Backend instance health, Capacity planning	Any Capacity (warning), 0 (critical)
istio_requests_total	Counter	Envoy (Prometheus mixer)	source_app, destination_service, mesh_version	Service mesh version, Downstream traffic	Downstream (warning)
keycloak_logins_total	Counter	Keycloak (metrics endpoint)	realm, client_id, status (success, error)	Authentication volume, Failure analysis (warning)	Failure rate > 5% (warning)
keycloak_user_sessions_active	Gauge	Keycloak (session store)	realm, client_id	Concurrent user sessions, Application sizing	Approaching limit (80% > 1000)
redis_connected_clients	Gauge	Redis (exporter)	Instance (master/replica)	Redis connection load, Capacity planning	> 1000 (warning), > 2000 (critical)
ssl_cert_expiry_timestamp	Gauge	cert-manager (metrics)	certificate_name, namespace	Certificate lifecycle management	< 30 days (warning), < 7 days (critical)
ratelimit_tokens_remaining	Gauge	Custom (Redis exporter)	Dimension (global/user/token/key), window	Rate limit tracking, Overage (info)	< 10 (info), < 0 (block)

8.2 Distributed Tracing

Distributed tracing captures the complete journey of a request as it flows through the gateway, mesh, and backend services, enabling root-cause analysis of latency issues and understanding of complex service interactions. The trace context propagates via W3C Trace Context headers (traceparent, tracestate) injected by Kong and carried by Istio through all downstream calls. Traces are sent to Jaeger (or compatible backend like Tempo/Grafana Cloud) with configurable sampling rates (100% for errors, 10% for slow requests, 1% for normal traffic in production).

Component	Tracing Setup	Span Attributes	Sampling Policy	Retention
Kong Gateway	OpenTelemetry plugin - https://github.com/kong/otel	http.method, http.route, http.status_code, http.error_code	Always sample errors (500, 502, 503, 504)	5 days (1 day per day limit)
Istio Sidecar (Envoy)	Built-in Jaeger tracer (enabled via mesh config)	source.workload_namespace, source.workload_name	Random, default 1% (configurable)	30 days (configurable)
Backend Services	OpenTelemetry SDK (language specific)	service.name, operation.name	Tracer parent (sampling)	30 days (configurable)
Keycloak (Auth)	Keycloak Quarkus OTEL integration	log.type, log.level, log.message	Always sample failures, MF, A events	30 days (configurable)

8.3 Alerting Rules and Runbooks

Alerting rules translate metric conditions into actionable notifications, ensuring that operators are informed of conditions requiring human intervention before they impact users. Alerts are defined in PrometheusRule CRDs (for Kubernetes-native workflow) with severity levels (critical, warning, info) that determine notification channels (PagerDuty for critical, Slack for warning, email for info). Each alert links to a runbook documenting investigation steps and remediation actions.

Alert Name	Severity	Expression (simplified)	Condition	Notification Channel	Runbook Summary
KongHighErrorRate	Critical	rate(kong_http_requests_total{status="500",...}) / rate(kong_http_requests_total{...}) > 0.05	Status of 500 (5%) or more	PagerDuty (on-call)	Check Kong logs for errors
KongLatencyP99High	Warning	histogram_quantile(0.99, rate(kong_latency_ms_total{...})) > 1.2	P99 latency > 1.2s	Slack (#infra-alerts)	Identify slow routes
AuthFailureSpike	Critical	increase(keycloak_logins_total{auth_failure="true"}) > 100	Auth failures > 100	PagerDuty (on-call)	Check for brute force
CertificateExpiringSoon	Warning	ssl_cert_expiry_timestamp - time() < 86400 * 7	Cert < 7 days to expire	Email (Security team)	Slack (#ops-perimeter)
RateLimitBreached	Info	ratelimit_tokens_remaining < 10	Global or "global" ant = 0	Slack (#infra-alerts)	Check rate limit config
RedisMemoryHigh	Warning	redis_memory_used_bytes / redis_memory_max_bytes > 0.85	Redis memory usage > 85%	Slack (#infra-alerts)	Check Redis config
MeshMTLSMismatch	Critical	istio_tcp_connections_opened{...}	Non-mTLS connection	PagerDuty (Security)	Identify source/dest namespace

9. IMPLEMENTATION ROADMAP

The following roadmap outlines the phased implementation approach for the API Gateway and Authentication architecture, sequencing work items to deliver incremental value while managing dependencies. The timeline assumes a dedicated team of 2 backend engineers, 1 DevOps/SRE engineer, and 0.5 security engineer working over approximately 10-14 weeks. Integration points with the Admin Interface (Phase 1A) and backend services are called out explicitly.

Phase	Duration	Deliverables	Dependencies	Success Criteria
Phase 1: Foundation (Weeks 1-2)	2 weeks	- Kong Gateway deployment (K8s) - Basic auth plugin (JWT validation) - Redis cluster for state - TLS termination + cert-manager - Health check endpoints	K8s cluster ready; DNS configured; Certificates issued	Gateway operational; TLS validated
Phase 2: Identity Provider (Weeks 3-5)	3 weeks	- Keycloak deployment (HA Postgres) - User import/migration scripts - OIDC client registrations - First-party login flow working - MFA (TOTP) enrollment and validation	Database schema finalized; User directory imported	Auth flow functional; MFA enforced
Phase 3: Gateway Hardening (Weeks 6-8)	3 weeks	- Complete plugin suite (ACL, rate limiting) - Rate limiting by tier (from LMS) - WAF rule deployment (ModSecurity CRS) - CORS policy configuration - Request/response transformation	Rate limit engine; WAF ruleset; LMS API available	Security policies active; Traffic protected
Phase 4: Service Mesh (Weeks 9-10)	2 weeks	- Istio installation (profile: demo) - Sidecar injection for api namespaces - mTLS (STRICT mode) for internal traffic - Kiali dashboard for visualization - Traffic policies (timeouts, retries)	K8s production; Sufficient TLS work for sidecars	Service mesh operational; mTLS enforced
Phase 5: Observability (Weeks 11-12)	2 weeks	- Prometheus scraping (Kong, Istio, Keycloak) - Grafana dashboards (gateway, auth, mesh overview) - Jaeger tracing (sampling configured) - AlertManager rules (this spec) - Runbook documentation	Infrastructure complete (this spec); Dashboards dev	Monitoring & Alerting functional
Phase 6: Testing & Launch (Weeks 13-14)	2 weeks	- Load testing (target: 10K RPS) - Penetration test (scope: gateway + auth) - Chaos engineering (kill pods, simulate failures) - Documentation (runbooks, API ref for auth) - Production cutover plan	Staging environment mirrors prod; Load test passes	System ready for production

9.1 Testing Strategy

Comprehensive testing validates that the gateway and authentication system meet functional requirements, performance targets, and security standards before production deployment. The testing strategy spans unit tests (individual plugin logic), integration tests (end-to-end flows through gateway), load tests (performance under stress), security tests (vulnerability assessment and penetration testing), and chaos tests (resilience under failure conditions).

Test Type	Tools	Scope	Coverage Target	Execution Timing	Owner
Unit Tests	pytest (Python for custom plugins); JUnit (Java); Jest (JavaScript); XCTest (iOS); JUnit (Android)	Individual code units, functions, and classes	100% code coverage; Critical paths 100%	Before deployment; Regularly	Backend developers
Integration Tests	Postman/Newman collections; Selenium (web); JMeter (API)	Multiple components, e.g., login flow, checkout process	End-to-end scenarios; Critical paths 100%	After development; Before production	QA Engineers
Load Tests	k6 (recommended); Locust; JMeter	Gateway throughput (RPS); Database performance	Sustained 10k RPS for 1 hour; 99.9% success rate	During development; Before production	Performance Engineers
Penetration Test	External vendor (e.g., Coalfire, NCC Group); OWASP ZAP (internal)	Security vulnerabilities, data breaches	Identify critical security flaws	Quarterly; Before major releases	Security Team
Chaos Tests	Chaos Monkey (Gremlin); Chaos Toolkit	System resilience, disaster recovery	Simulate failures, e.g., node loss, network issues	Regularly in staging environment	DevOps Engineers

APPENDIX A: CONFIGURATION REFERENCE

This appendix provides quick-reference configuration snippets for the key components described in this specification. These examples illustrate typical production configurations but should be adapted to specific deployment requirements, organizational policies, and environment constraints. All sensitive values (keys, passwords, secrets) should be externalized to Vault or Kubernetes Secrets and never committed to version control.

Component	Configuration File/Location	Key Parameters	Example Snippet (redacted)
Kong Gateway (kong/kong)	/etc/kong/kong.conf (ConfigMap)	workers, pg_host, redis_addr, plugins	workers = 1 pg_host = postgres.cybersoc.svc.cluster.local redis_addr = redis.cybersoc.svc.cluster.local:6379 plugins = bundled, custom-audit, custom-acl, custom-ten
Kong Declarative Configuration	Kubernetes ConfigMap (secrets, routes, plugins)	services, routes, plugins (per-route/global)	"services": [{"name": "cybersoc-api", "host": "api-backend"}] "routes": [{"name": "api-v1", "service": "cybersoc-api", "paths": ["/api/v1"]}] "plugins": [{"name": "jwt", "config": {...}}, {"name": "acl", "config": {...}}]
Keycloak (realm settings)	Keycloak Admin Console	Enabled, Token Lifespan, Refresh Token Lifespan, Theme, SMTP	Token Lifespan: 900 (15 min) Refresh Token Lifespan: 604800 (7 days) Theme: cyberSOC (custom) SMTP: smtp-relay.cybersoc.internal:587
Keycloak (Client configuration)	Keycloak Admin Console	Valid Redirect URIs, Web Origins, Client Auth, Protocol Mappers	Valid Redirect URIs: https://app.cybersoc.platform/* Client Auth: client_secret_post Protocol Mapper: tenant_id (from user attr)
Istio (mesh config)	istio-system namespace (Helm Operator CRDs)	meshConfig, accessLogFile, defaultConfig, tracing, sampling, proxyStatsMatcher	kind: IstioOperator spec: meshConfig: accessLogFile: /dev/stdout defaultConfig: tracing: sampling: 100.0 proxyStatsMatcher: inclusionPrefixes: [...]
Prometheus (scrape config)	prometheus.yml	Scrape intervals, target discovery, relabeling	- job_name: 'kong' kubernetes_sd_configs: [...] relabel_labels: [__meta_kubernetes_pod_label_app=kong] scrape_interval: 15s
AlertManager (routing)	alertmanager.yml	Grouping tree, group_by, receivers, inhibit rules	- receiver: 'pagerduty-critical' match: severity: critical - receiver: 'slack-warning' match: severity: warning

APPENDIX B: ERROR RESPONSE SCHEMA

All API errors follow RFC 7807 Problem Details for HTTP APIs format, providing machine-readable error structure that enables programmatic error handling by client applications. This appendix documents the standard error response schema, common error types, and recommended client-side handling strategies for each error category.

Field	Type	Required	Description	Example
type	URI (string)	Yes	Error type identifier (link to doc urn:cybersoc:error:invalid-token)	"urn:cybersoc:error:invalid-token"
title	string	Yes	Human-readable error summary	"Invalid or expired authentication token"
status	integer	Yes	HTTP status code (matches response)	401
detail	string	No	Specific error details (may include full error message)	"Token expired at 2024-08-24T15:30:00Z; Issuer mismatch"
instance	URI (string)	No	Reference to specific resource or error code for lookup	"/errors/abc123def456"
trace_id	string (UUID)	Yes (tracing enabled)	Correlation ID for log/tracing lookup	"abc3d4-e5f6-7890-abcd-ef1234567890"
timestamp	datetime (ISO 8601)	Yes	When the error occurred	"2024-08-24T15:30:05.123Z"
errors	array (object)	No (validation errors only)	List of field-level errors (for 400 Bad Request)	["field": "email", "message": "Invalid email format"]
retry_after	integer (seconds)	No (rate limited)	Seconds until client should retry	60
documentation	URI (string)	No	Link to help documentation for this error	"https://docs.cybersoc.platform/errors/E001"

Error Type URN	HTTP Status	Title	When Returned	Client Action
urn:cybersoc:error:invalid-token	401	Invalid or Expired Token	Token signature invalid; expired; Clear stored token, Redirect to login	Ob
urn:cybersoc:error:insufficient-scope	403	Insufficient Permissions	JWT lacks required scope for requested resource	Request user for missing permission; Guide to roles
urn:cybersoc:error:rate-limit-exceeded	429	Rate Limit Exceeded	Client exceeded configured rate limit	Wait (Retry after seconds); Implement exponential backoff
urn:cybersoc:error:quota-exhausted	429	Quota Exhausted	Tenant/user consumed entire allocated quota	Monitor quota (dashboard shows when); Request increase
urn:cybersoc:error:validation-failed	400	Validation Error	Request body fails schema validation	Display field-level error messages; Type/formatting hints
urn:cybersoc:error:not-found	404	Resource Not Found	Requested resource (user, tenant, document) does not exist	Verify resource ID; Handle 404 gracefully
urn:cybersoc:error:conflict	409	Conflict	Create with duplicate unique field address or attempt to modify existing state	Check resource state; Merge changes; Retry with idempotency
urn:cybersoc:error:upstream-service-unavailable	502/503/504	Upstream Service Unavailable	Backend service unavailable; Temporary outage	Display friendly message; Offer to try again later
urn:cybersoc:error:mfa-required	401	MFA Required	User enrolled in MFA but didn't provide second factor	Prompt for MFA factor; Stop Authn; Redirect to MFA app
urn:cybersoc:error:account-locked	403	Account Locked	Too many failed attempts; Admin flagged suspicious activity	Show lock status; Suggest password reset; Contact support

APPENDIX C: PRE-LAUNCH SECURITY CHECKLIST

This checklist must be completed and signed off by the security team before the API Gateway and Authentication system can be deployed to production. Items are organized by criticality, with all CRITICAL items requiring explicit sign-off. WARNING items should be addressed or formally accepted as risk with documented mitigation timeline. INFO items represent best practices that strengthen security posture but are not blocking for launch.

#	Item	Criticality	Status	Sign-off	Evidence/Notes
1	TLS 1.2/1.3 only (no SSLv3/TLS 1.0/1.1)	CRITICAL	☒ Pass ☐ Fail ☐ Waived	✓	Cipher scan results (SSL Labs A+ grade)
2	HSTS header enabled with max-age >= 31536000 (1 year)	CRITICAL	☒ Pass ☐ Fail ☐ Waived	✓	curl -I output showing Strict-Transport-Security
3	All authentication endpoints enforce MFA for admin roles	CRITICAL	☒ Pass ☐ Fail ☐ Waived	✓	Test login with admin account; verify MFA prompt
4	JWT signing keys rotated within last 90 days; Old keys reclaimed	CRITICAL	☒ Pass ☐ Fail ☐ Waived	✓	Keycloak keys list showing issue/expiration date
5	Rate limiting enforced for all endpoints (including auth)	CRITICAL	☒ Pass ☐ Fail ☐ Waived	✓	Load test showing 429 responses after threshold
6	CORS policy restrictive (no wildcard * for credentials)	CRITICAL	☒ Pass ☐ Fail ☐ Waived	✓	OPTIONS preflight response headers; Cross-origin
7	WAF enabled with OWASP CRS (Paranoia Level 1 minimum)	CRITICAL	☒ Pass ☐ Fail ☐ Waived	✓	WAF dashboard showing rules loaded; Test attack
8	Security headers complete (CSP, X-Frame-Options, X-Content-Type-Options, etc.)	CRITICAL	☒ Pass ☐ Fail ☐ Waived	✓	securityheaders.com scan or curl -I header inspection
9	Error messages sanitize internal details (stack traces, SQL IDs)	CRITICAL	☒ Pass ☐ Fail ☐ Waived	✓	Trigger various errors; Inspect response bodies
10	Audit logging enabled for all auth events and admin actions	WARNING	☒ Pass ☐ Fail ☐ Waived	✓	Elasticsearch query showing events for test actions
11	Brute force protection active (account lockout after N failed logins)	WARNING	☒ Pass ☐ Fail ☐ Waived	✓	Failed login test (N+1 times); Account locked; Timeout
12	Session management: Secure; HttpOnly; SameSite cookies	WARNING	☒ Pass ☐ Fail ☐ Waived	✓	Browser DevTools > Application > Cookies; Inspect
13	Dependency vulnerability scan completed (0 CRITICAL/HIGH findings)	WARNING	☒ Pass ☐ Fail ☐ Waived	✓	Trivy/Snyk scan report; Remediation or acceptance
14	Penetration test completed (0 CRITICAL/HIGH findings)	CRITICAL	☒ Pass ☐ Fail ☐ Waived	✓	Pen test report executive summary; Findings remediated
15	Disaster recovery tested (failover to DR site < 5 minutes)	INFO	☒ Pass ☐ Fail ☐ N/A	✓	ADR test runbook; Timestamps of failover/failback
16	Incident response playbook exists and team trained	INFO	☒ Pass ☐ Fail ☐ N/A	✓	Playbook document link; Training attendance record